

AI 调试对决实验报告

团队：第67组 | 阶段：Phase 4 编码开发 | 日期：2026-06-01

关键实验二 ----- "AI 调试对决"

团队在集成过程中选择 2 个有代表性的 Bug，分别记录纯人工调试和 AI 辅助调试的完整过程，比较两者的差异。每个 Bug 覆盖六个步骤：问题描述、定位耗时、定位准确度、修复方案、方案质量、最终修复。

Bug #1：删除评价后用户平均分计算为 null / NaN

问题描述

管理员删除某用户的唯一一条评价后，该用户的信用页显示 NaN（Not a Number）或 null。前端页面渲染异常，评分星星组件显示空白或报错。

6 步对比表

步骤	纯人工调试	AI 辅助调试
1. 问题描述	开发者看到前端报 NaN，自己的第一反应是"前端计算有问题"，打开浏览器 DevTools 打断点追踪 creditScore 的计算链路，发现 averageScore 字段为 null。	将以下信息一并提供给 AI：（1）前端截图显示 NaN；（2）数据库中 average_score = null, score_num = 0 的记录截图；（3）ReviewService.java 中 recalculateUserAverageScore 方法和 deleteReview 方法的完整代码。
2. 定位耗时	约 25 分钟 。从怀疑前端 → 查数据库 → 读后端代码 → 加日志打印 SQL 执行顺序 → 排查 JPA 一级缓存 → 阅读 Hibernate flush 机制文档。	约 5 分钟 。AI 在读完代码后直接指出："DELETE 语句在 @Transactional 方法内不会立即 flush 到数据库，后续 SELECT AVG 可能读到已删除的数据，导致 newCount = 0 但 newAvg != null 的矛盾状态。"
3. 定位准确度	最终定位准确，但走了弯路：先错误怀疑了前端计算逻辑和 MySQL AVG 函数在空结果集上的行为，排除后才锁定 JPA flush 时机问题。	准确 。AI 直接定位到 JPA 持久化上下文的刷新时机问题，并同时指出了代码中缺少防御性判断（当 newCount = 0 时没有优先处理）。
4. 修复方案	人工方案：在 deleteReview 中 reviewRepository.delete() 之后加一行 reviewRepository.flush()，强制立即执行 DELETE。	AI 方案：（1）同样建议加 flush()；（2）额外建议增强 recalculateUserAverageScore 的防御性——先判断 newCount == null
5. 方案	人工方案的优点是改动最小、见效快；缺点是只修复了 deleteReview 这一条调用路径，如果	AI 方案更 全面 ——flush 解决了当前 Bug 的根因，防御性判断作为第二道防线确保无论何种调用路径

步骤	纯人工调试	AI 辅助调试
案质量	其他地方也调用了 recalculateUserAverageScore 且存在同样的 flush 时机问题，仍会复现。	都不会返回 null。不引入新问题，属于"治标又治本"。
6. 最终修复	全部采纳 AI 方案。 理由：AI 的方案比人工的更完善，防御性判断代价极小（两行代码），但能一劳永逸避免同类问题。人工加 flush 的思路方向正确但不够全面。	

Bug #2：系统通知点击后未读红点不消失

问题描述

用户进入消息页面后，系统通知 Tab 上显示未读红点数字。用户逐条点击通知阅读内容，但 Tab 上的未读红点始终不减少，通知项的紫色未读高亮背景也不消失。即使刷新页面等待 10 秒轮询刷新后，未读依然存在。

6 步对比表

步骤	纯人工调试	AI 辅助调试
1. 问题描述	开发者打开浏览器 DevTools Network 面板，逐条点击系统通知， 发现没有网络请求发出。 然后检查页面源代码，发现系统通知列表项没有 @click 事件。再去对比私信会话列表，发现私信有点击事件 @click="selectConversation(conv)" 且内部调用了 PUT /messages/read-conversation。	将以下信息提供给 AI：（1）系统通知点击后红点不消失的现象描述；（2）MessagesPage.vue 完整代码；（3）询问"为什么系统通知点击后没有标记已读，而私信会话点击后可以标记已读？"
2. 定位耗时	约 10 分钟 。从怀疑后端接口 → 检查 Network 发现无请求 → 检查前端模板发现无 @click → 对比私信逻辑确认缺少事件绑定。	约 2 分钟 。AI 在阅读 MessagesPage.vue 后直接指出：（1）系统通知的 notification-item 没有 @click 事件；（2）对比私信 conversation-item 的 @click="selectConversation(conv)";（3）后端已提供 PUT /messages/read/{id} 接口但前端从未调用。
3. 定位准确度	定位准确，但排查路径不够高效：先花时间怀疑后端 + 数据库，排除后才转向前端模板代码。	准确 。AI 同时检查了前后端代码，直接发现了三处关联问题：模板缺少事件、script 缺少处理函数、后端接口已存在但未被调用。
4. 修复	人工方案：为系统通知项绑定 @click，新增处理函数调用 PUT /messages/read/{id}，标记已读后更新本地状态。	AI 方案：（1）模板加 @click="handleNotificationClick(notif)";（2）新增 handleNotificationClick 函数，调用 /messages/read/{id} 并即时更新本地 notif.read =

步骤	纯人工调试	AI 辅助调试
方案		true；(3) CSS 中将 cursor: default 改为 cursor: pointer 提示可点击；(4) 建议对已读通知做判断，避免重复请求。
5. 方案质量	人工方案的优点是直接解决问题；缺点是可能遗漏细节——比如已读状态只等 10 秒轮询刷新、没有即时更新本地状态导致 UI 延迟反馈、鼠标样式没有提示可交互。	AI 方案更 细致 ：涵盖了即时本地状态更新（不等轮询）、优化用户体验（cursor 样式）、避免冗余请求（已读判断）。不引入新问题，方案完整可落地。
6. 最终修复	全部采纳 AI 方案 。理由：AI 不仅修了 Bug，还同步优化了用户体验细节（cursor、即时更新），方案比人工更全面，且改动范围仅限一个文件，风险可控。	

实验记录汇总表

Bug #	Bug 描述	纯人工定位耗时	AI 辅助定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
1	删除评价后用户平均分计算为 null / NaN（JPA 事务 + 空值处理）	~25 min	~5 min	准确	可用且更全面	AI 方案
2	系统通知点击后未读红点不消失（前端事件绑定缺失）	~10 min	~2 min	准确	可用且更细致	AI 方案

分析问题

AI 在哪类 Bug 上表现较好？

- 框架机制类 Bug**（如 JPA flush 时机、事务传播）：AI 对常见框架的底层机制非常熟悉，能快速指出 Hibernate 一级缓存和 @Transactional 的行为特征，这类知识对初级开发者来说需要大量文档阅读才能掌握。
- 前后端对比类 Bug**（如前端缺少调用 vs 后端接口已就绪）：AI 能同时审阅前后端代码，快速建立"接口已存在但未被调用"的映射关系，人工做同样的对比需要在多个文件间反复跳转。
- 代码遗漏类 Bug**（如缺少 @click、缺少字段）：AI 在扫描模板和 script 时能自动比照相似功能的写法（如私信 vs 系统通知），发现不一致。

AI 在哪类 Bug 上表现较差？

- UI 组件库边界行为**（如 Element Plus 的 el-rate 组件在值为 0 时不触发 change 事件）：AI 倾向于按标准文档行为给出建议，对特定组件版本的"非标准"行为缺乏敏感度，需要人工补充组件库的已知问题信息。
- 需要实际运行环境才能复现的 Bug**（如端口冲突、环境变量缺失、MySQL 版本差异）：AI 只能分析代码，无法感知运行时环境的具体状态，这类问题仍需人工排查环境。

AI 是否出现只修表象、不找根因的情况？

在本次实验的 2 个 Bug 中，**未出现**只修表象的情况。AI 在两个案例中都定位到了真正的根因：

- Bug #1：AI 没有简单地建议"给 averageScore 加个 ifNull 兜底"，而是深入到 JPA flush 机制层面解决 DELETE 和 SELECT 的执行顺序问题，同时再加防御性判断作为第二道保险。
- Bug #2：AI 没有只建议"手动刷新页面"，而是从模板缺少事件绑定这一根因入手，提出了完整的前端修复方案。

不过，这在很大程度上得益于**提供了充分的上下文**（完整代码文件 + 明确的复现步骤）。如果上下文不完整，AI 可能会给出表面级的修补建议。

人工提供哪些上下文后，AI 的调试建议明显变好？

1. **完整代码文件**（而非仅报错信息）：提供完整的 .vue / .java 文件让 AI 能建立前后端对照视图，效果远超过仅提供报错日志。
2. **明确的复现步骤**（1→2→3 的操作序列）：帮助 AI 在脑中模拟执行路径，精确定位触发 Bug 的代码行。
3. **相似功能的对比参照**（如"私信点击能标记已读，系统通知不能"）：这种对比描述能引导 AI 去查找两段代码的差异，快速锁定遗漏点。
4. **数据库实际状态**（如查询结果截图）：对后端 Bug 尤其关键，帮助 AI 区分"代码逻辑问题"还是"数据状态异常"。

结论

本次 "**AI 调试对决**" 实验选取了 2 个不同层面的 Bug（后端 JPA 事务机制 + 前端事件绑定缺失），AI 辅助调试在两个案例中的定位速度分别比纯人工快 **5 倍** 和 **5 倍**，且修复方案均被采纳。AI 的优势在于能快速跨越前后端代码建立全景视图、对框架底层机制有系统性的知识储备；局限在于无法感知运行时环境和 UI 组件库的非标准行为。提供充分的上下文（完整代码 + 复现步骤 + 对比描述）是获得高质量 AI 调试建议的关键前提。